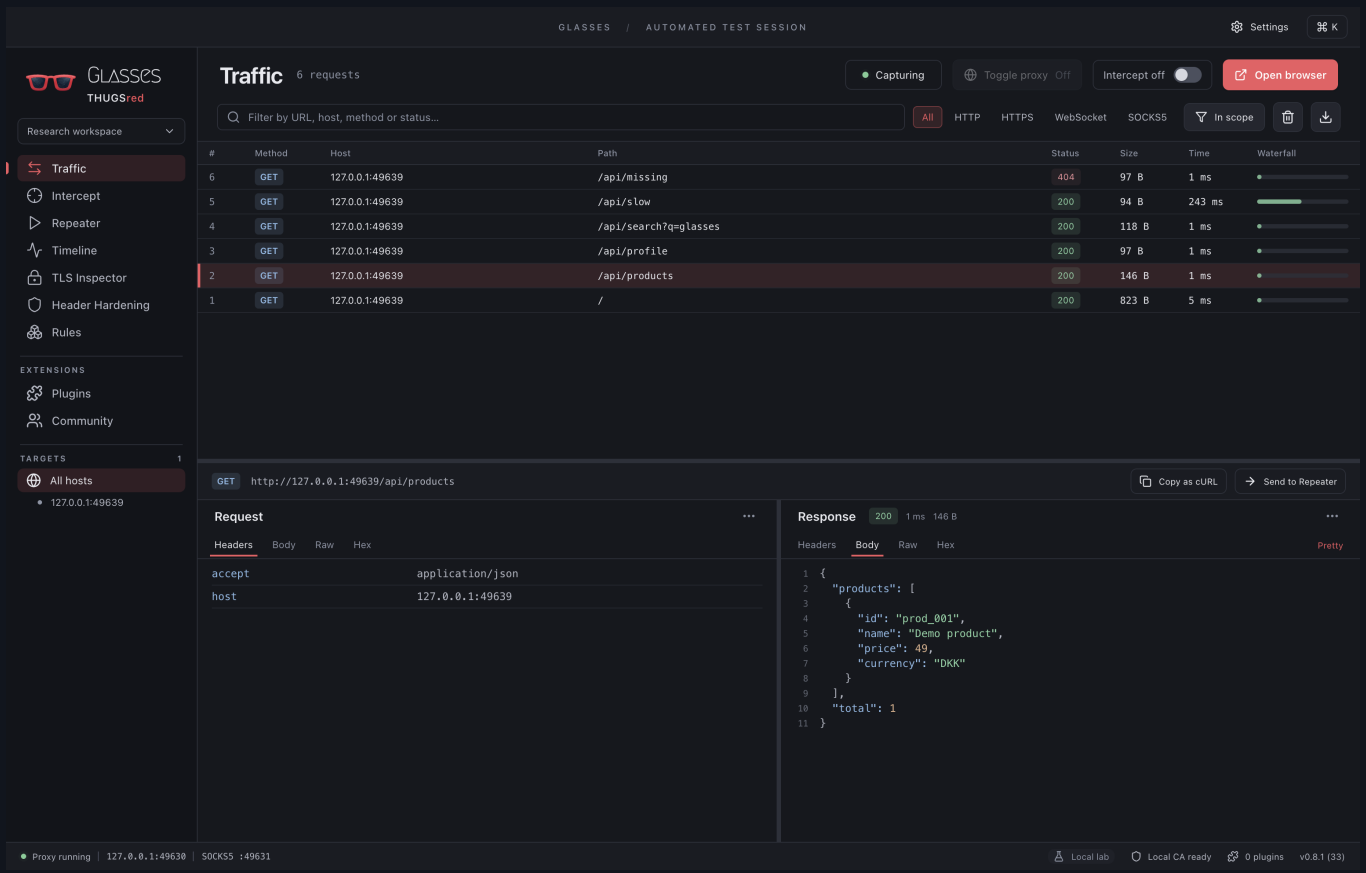


Glasses

Look closer.

The illustrated field manual.
Capture. Inspect. Intercept. Repeat. Extend.



DOCS 1.0.0 (1) / APP 0.8.1 (33)

glasses.thugs.red

Real application screenshot using the built-in local lab. macOS development release.

START HERE

Your guide to Glasses

A practical reference for researchers, web developers and extension authors. The screenshots use local lab traffic and example settings, never private browsing data.

03 First run and local lab
04 The research workspace
05 Reading captured traffic
06 Context menus and transforms
07 Intercept, edit and forward
08 Repeater and request editing
09 Iterations and fuzz payloads
10 Site Timeline / Speed Inspector
11 SSL / TLS Inspector
12 Header Hardening and rules
13 System proxy toggle
14 Proxy and SOCKS relay chains
15 Privacy bypass lists
16 Organization CA and trust
17 Themes, storage and exchange
18 Installing community extensions
19 Building an extension
20 Security architecture
21 Build and release reference
22 Troubleshooting and glossary

What this release is

Glasses 0.8.1 (33) is a usable local development release. macOS ARM64 is the primary tested desktop. Packages target Intel macOS and x64/ARM64 Windows/Linux; those desktops still need platform acceptance. Signed distribution, notarization and enterprise deployment controls remain release work.

Official home

glasses.thugs.red is the official application and community address. The app supports opt-in catalog browsing, signed package review and explicit installation. The service is deployed separately; local JSON installation works offline. Automatic updates are not implemented.

How to read the screenshots

Ports, fingerprints and request durations are examples. Your installation creates its own CA and local lab port. Dark and light modes show the same tools. The guidance describes implemented behavior; the release reference marks important limitations.

GETTING STARTED

First run and local lab

Start with real local traffic before connecting your own browser or corporate network.

1 Open the application

On Apple Silicon, use the arm64 macOS package; on Intel, use x64. Open the DMG and copy Glasses to Applications, or extract the ZIP. These local builds are not Developer ID signed or notarized. Use your organization's normal approval process for development software.

2 Try Local lab

Click **Local lab** in the footer. Glasses starts its listeners and requests six routes from a server on your own Mac: a page, JSON APIs, a slow response and a missing route. Select a row to inspect its request and response.

3 Try an experiment

Send an API request to Repeater. Change a query value, send it again, then compare the new response. Open Timeline to see the deliberately slow lab route. This checks your basic workflow without an external target.

4 Connect a browser

Open browser starts an isolated Chromium research session. On a fresh launch it explicitly uses Glasses and accepts correctly named, currently valid certificates signed by the active research CA. It does not share your personal browser profile.

5 Connect other clients

The default HTTP/HTTPS endpoint is **127.0.0.1:8080**; SOCKS5 is **127.0.0.1:1080**. Export the public CA if your client needs to trust intercepted HTTPS. The proxy listens on loopback only.

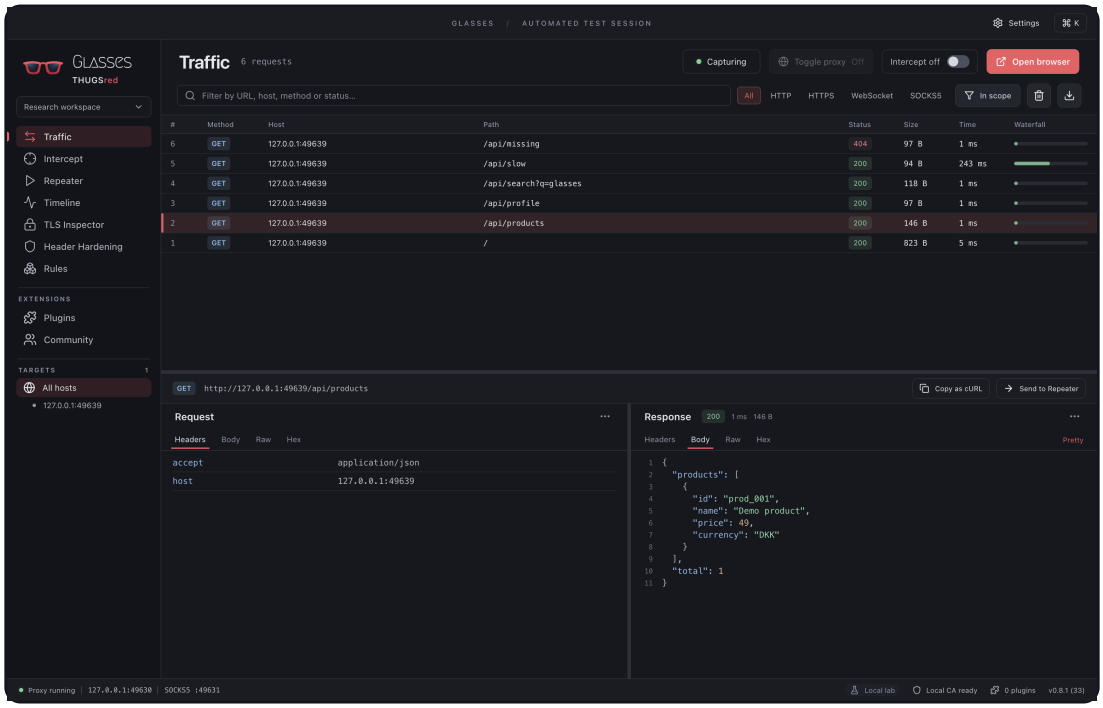
Before corporate use

Configure the upstream relay first when your network requires a proxy. Add confidential hosts to the privacy bypass list before opening them. Use **Toggle proxy** for system-aware clients; the research browser also works without global changes.

WORKSPACE

The research workspace

The toolbar controls routing and interception; the sidebar moves between the questions you are investigating.



Workspace overview: local lab records, host navigation and separate request/response inspection.

Navigate by task

Traffic is the capture list and paired inspectors. **Intercept** holds paused messages. **Repeater** sends edited requests and runs payload experiments. Timeline, TLS Inspector and Header Hardening provide focused analysis.

Settings stay separate

Use the top-right Settings button or **Cmd/Ctrl+,**. All themes, network listeners, interception filters, relays, bypass lists, CA choices, storage and community preferences live in that pane.

Know the three switches

Start proxy / Capturing controls the local listeners. **Toggle proxy** changes system routing on macOS. **Intercept** pauses matching messages. A running listener does not mean every application is using it.

Move quickly

Cmd/Ctrl+K opens the command palette. **Cmd/Ctrl+R** sends the selected record to Repeater when focus is outside an editor. Escape closes a context menu. The footer shows listener and CA state.

CAPTURE

Reading captured traffic

Preserve context while moving from a request list to the exact headers and bytes that explain it.

Find the relevant request

Filter by URL, host, method or status, choose a protocol filter, or select a host in the sidebar. **In scope** uses literal host/URL fragments from General settings. Scope is a display filter, not a privacy exclusion.

Choose the representation

Headers show fields. Body decodes supported content encodings and can pretty-print JSON. Raw combines a reconstructed HTTP/1.1 start line, headers and body. Hex shows bytes with offsets. Raw is not a wire-perfect TCP transcript.

Move to the right tool

Right-click a record to send it to Repeater, intercept future matching requests, view timing, inspect TLS on HTTPS targets, check response headers, or copy a URL/cURL command. Unavailable actions are disabled.

Respect capture boundaries

Captured bodies default to 2 MiB. Responses above the limit continue streaming; only the prefix is retained. Oversized incoming requests are rejected. A response that exceeds the limit cannot wait for whole-body interception.

Preview versus export

Text and hex previews are bounded so large data does not overwhelm the interface. Hex displays up to 64 KiB. Save the body to obtain all captured bytes. This still excludes bytes that were never retained because the capture limit was exceeded.

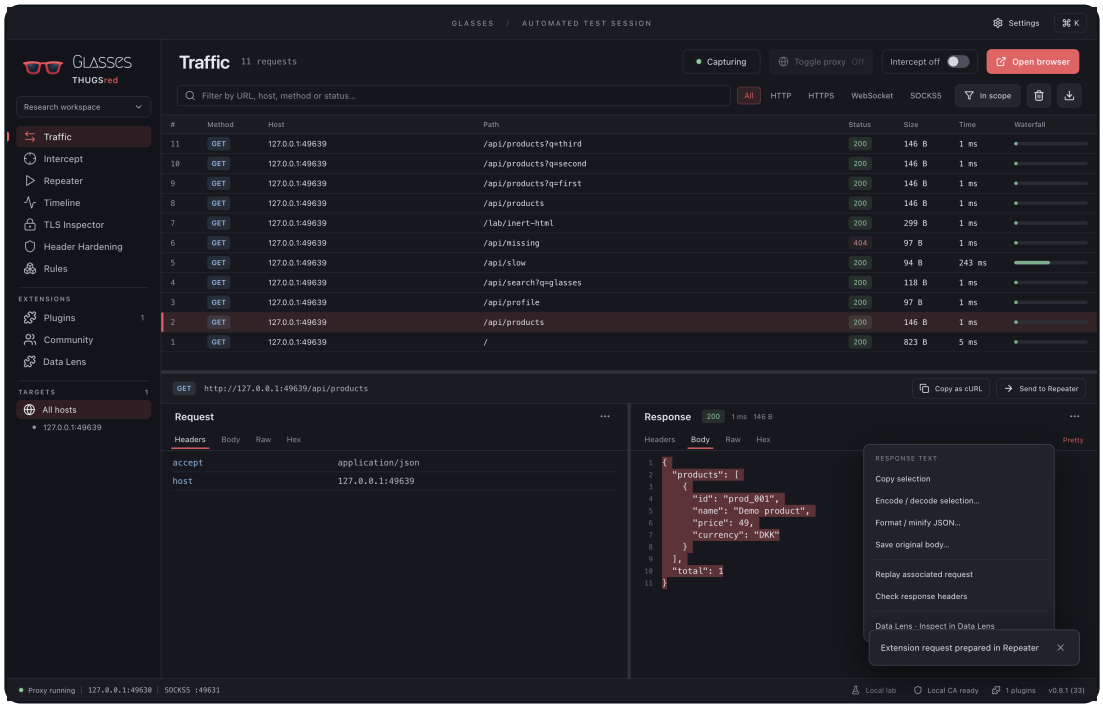
History is temporary

Records stay in memory until you export. Old completed entries are trimmed by the configured count and an approximately 128 MiB body budget. Clearing the list requires resolving paused messages first. Export before clearing if you need the evidence.

CONTEXT ACTIONS

Right-click where you work

Menus are specific to the view and selected data. Extensions can add commands to those same surfaces.



Response context menu with the Data Lens extension. Commands act on the originating record and selected text.

Body and byte views

Select a substring before opening the menu, or operate on the whole current view. Copy the selection/view, open the encode/decode preview, save the captured body, or transfer the related request to another tool.

Transform with a preview

Use URL, Base64, hexadecimal, HTML entity and JSON transformations. Check the output before copying it or replacing an editable selection. Invalid input reports an error. Previewing a captured body does not rewrite its stored evidence.

Headers

Header menus offer field-aware copy and rule actions alongside request handoffs. A response security header can lead directly to a hardening check. Transport-managed fields are protected from rewrite rules.

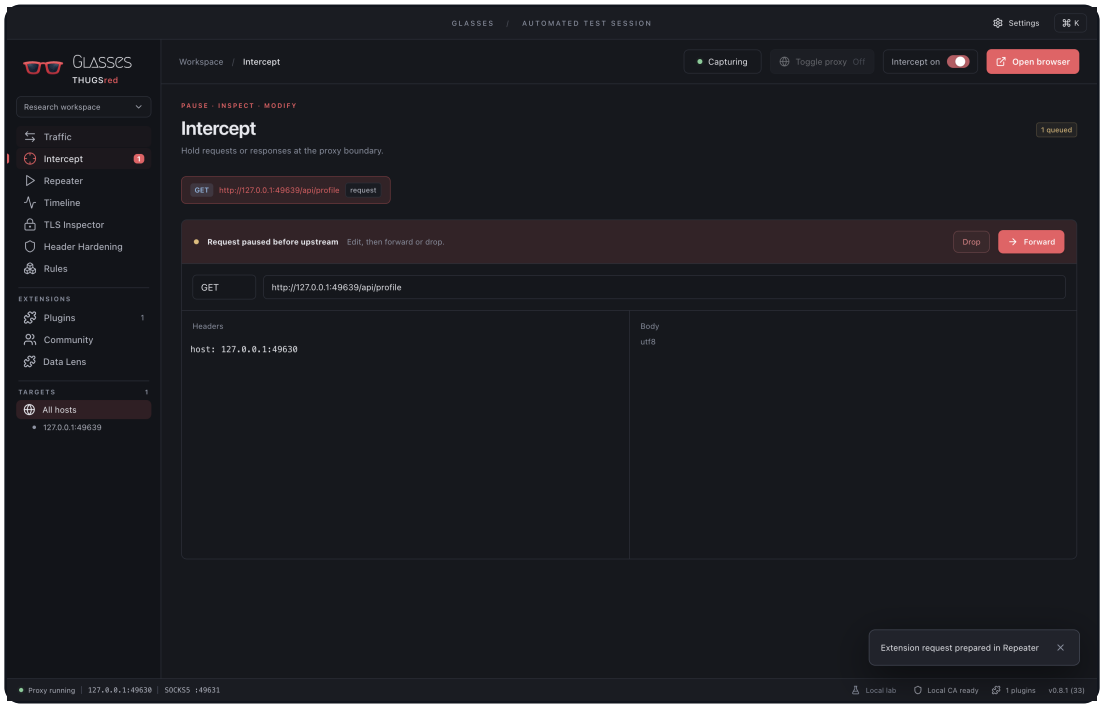
Community contributions

A plugin can register commands for request/response bodies and headers, traffic, Repeater, Intercept, Timeline and TLS. The menu labels include the plugin name. Its declared capabilities determine which selected data it receives.

INTERCEPTION

Intercept, edit and forward

Pause a matching request or response, make a controlled change, and choose what happens next.



An actual local request paused before forwarding. The editor and decision buttons stay together.

Configure the pause

In Proxy & interception, choose **Request**, **Response** or **Request and response**. Optionally restrict interception to a literal, case-sensitive URL fragment. Enable Intercept from the workspace toolbar.

Edit the pending message

Select a pending item. For requests, edit method, URL, headers and body. For responses, edit status, headers and body. **Forward** continues with your changes; **Drop** fails that exchange.

Keep the queue moving

The queue is limited to 100 pending messages. An unresolved message is dropped at the configured deadline, default 120 seconds. Turning Intercept off forwards pending items. Stopping the proxy drops them.

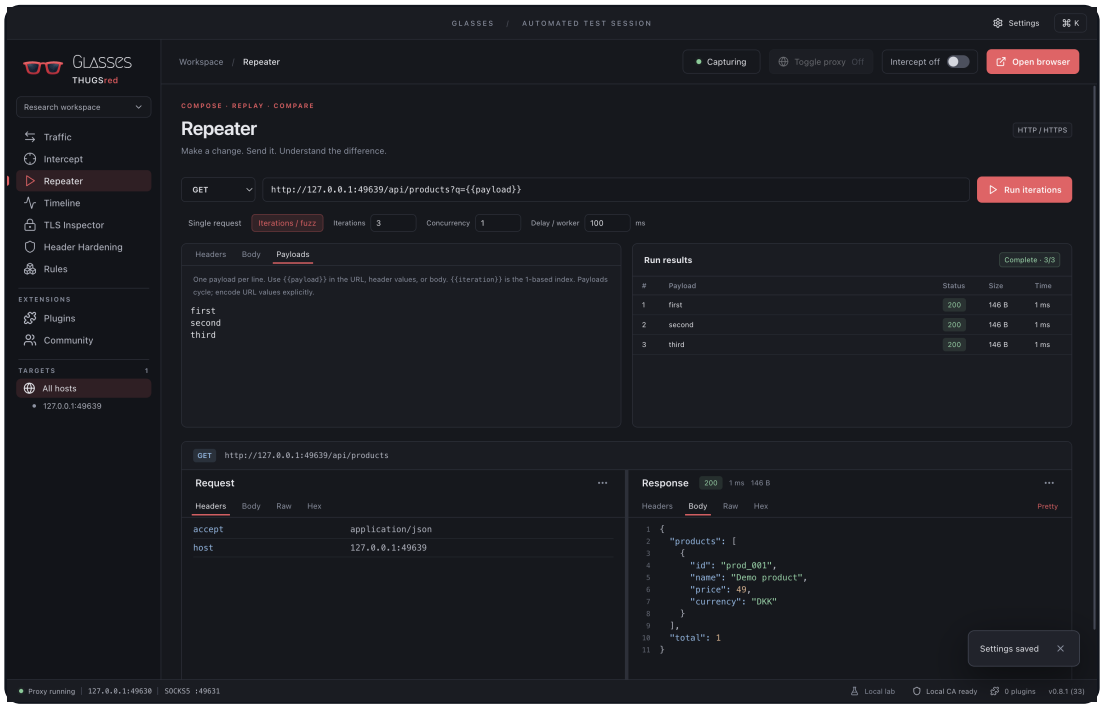
Know the boundaries

Large responses stream when they exceed the capture budget. Repeater requests do not enter the interactive interception queue. Bypassed destinations cannot be inspected; changing network/bypass settings disconnects tunnels and cancels pending work.

REPEATER

Repeat with intent

Rebuild an exchange, change one variable, and inspect what the server actually returns.



Repeater with a completed three-request payload run against the local lab.

Seed and send

Choose **Send to Repeater** on a captured HTTP request, or prepare a request draft from an extension. Review the method and URL. Edit headers as name/value lines; use the body editor for content. Click **Send request**.

Preserve the bytes you need

Use UTF-8 for ordinary text and Base64 for binary payloads. Glasses derives Host and Content-Length for the target and body. Header newlines and invalid method/URL forms are rejected.

Use the same route

Repeater uses the saved relay chain, upstream TLS verification and rewrite rules. The result appears in the session history and response pane. It cannot bypass a failed relay or a privacy exclusion silently.

A good debugging loop

Start with a successful baseline. Change a parameter or header, send, compare status/body/size, then save the useful records. Copy as cURL when you need a command-line reproduction. Repeater drafts do not send until you ask.

REPEATER

Iterations and fuzz payloads

Turn a single draft into a bounded experiment with explicit payloads, concurrency and timing.

Build a template

Enable **Iterations / fuzz**. Put `{{payload}}` in the URL, a header value or body. Put `{{iteration}}` where you want the one-based request number. Replacement is literal.

Define the run

Enter one payload per line. Payloads cycle if the iteration count is larger than the list. The maximum is 1,000 iterations and 10 concurrent workers. The delay is applied per worker, so concurrency affects aggregate request rate.

Example

URL:
`https://app.example/search?q={{payload}}`

Payloads: `alpha`, `beta`, `gamma`.
Set 3 iterations and concurrency 1 for one request per value. URL-encode payloads explicitly when needed.

Read the result table

Each result reports iteration, payload, status, captured record, duration or an error. Open a result to inspect its request and response. A failed request does not mean the whole run stopped; review the per-result error.

Cancel predictably

Cancel stops active requests and wakes waiting workers. Stopping the proxy or changing routing cancels active runs too. A server may already have received an earlier request; cancellation cannot undo server-side changes.

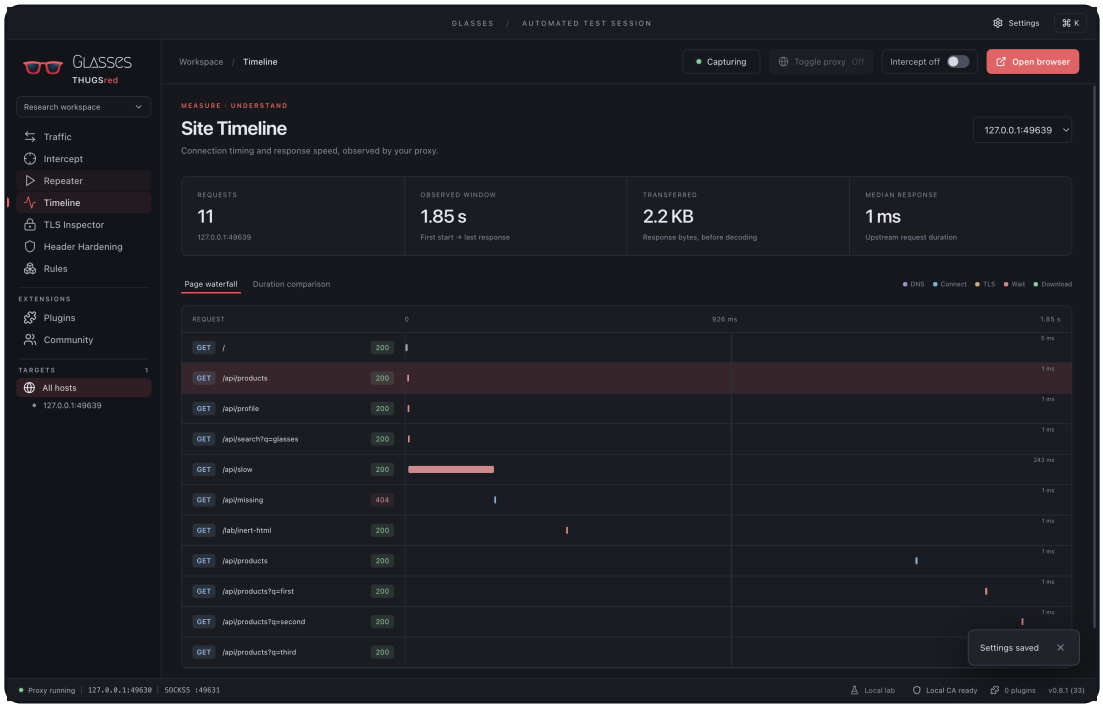
What fuzz means in this release

This is payload substitution and repeatable request execution. It is not an automatic vulnerability scanner, a combinatorial attack engine or a malformed-wire request-smuggling tool. Keep your interpretation tied to the actual response evidence.

PERFORMANCE

Site Timeline / Speed Inspector

See where observed network time is spent, compare slow requests, and follow an outlier back to its evidence.



The local lab includes a deliberate slow response so the waterfall has an explainable comparison.

Start with captured traffic

Timeline uses the current session. Capture a page or API workflow, then open Timeline. Compare request starts, total duration and transfer sizes; sort or inspect interesting rows and send them to Repeater.

Read measured phases

Connection, TLS, first-byte wait and download are based on proxy observations. A configured relay can add negotiation time. Missing phases are unavailable rather than invented; local lab transfers often have little measurable connection delay.

Use a repeatable comparison

Compare the same route and request under similar conditions. Run sequentially when measuring single-request latency; use concurrency deliberately when studying contention. Include response size and errors when comparing results.

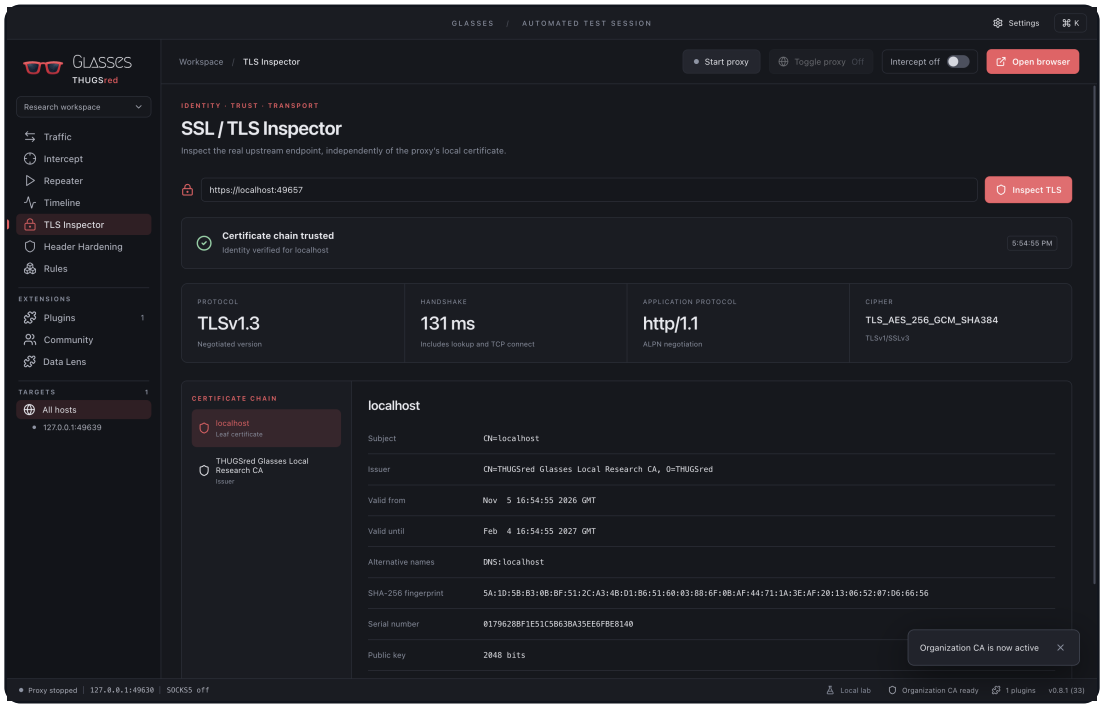
This is network timing

Glasses does not measure DOM rendering, browser layout, paint, JavaScript execution or Core Web Vitals. It does not replace a browser performance profiler. Export timing data or captures to continue analysis elsewhere.

TRANSPORT

SSL / TLS Inspector

Probe the upstream identity and transport independently of the certificate Glasses presents to a client.



A real TLS handshake with the local HTTPS lab. Certificates and timestamps vary per installation.

Inspect an endpoint

Enter an HTTPS host/URL and optional port, or open the inspector from a captured HTTPS record. Click **Inspect TLS**. The probe uses the saved relay chain and respects privacy exclusions.

Interpret the trust result

The inspector reports a validation error so you can examine a broken certificate. Ordinary proxied HTTPS still requires upstream validation. Inspecting a failure does not grant it trust for later browsing.

Read the chain

Select the leaf or an issuer to inspect subject, issuer, SANs, validity, fingerprint and certificate details. Copy or export certificate information when comparing deployments or reporting a mismatch.

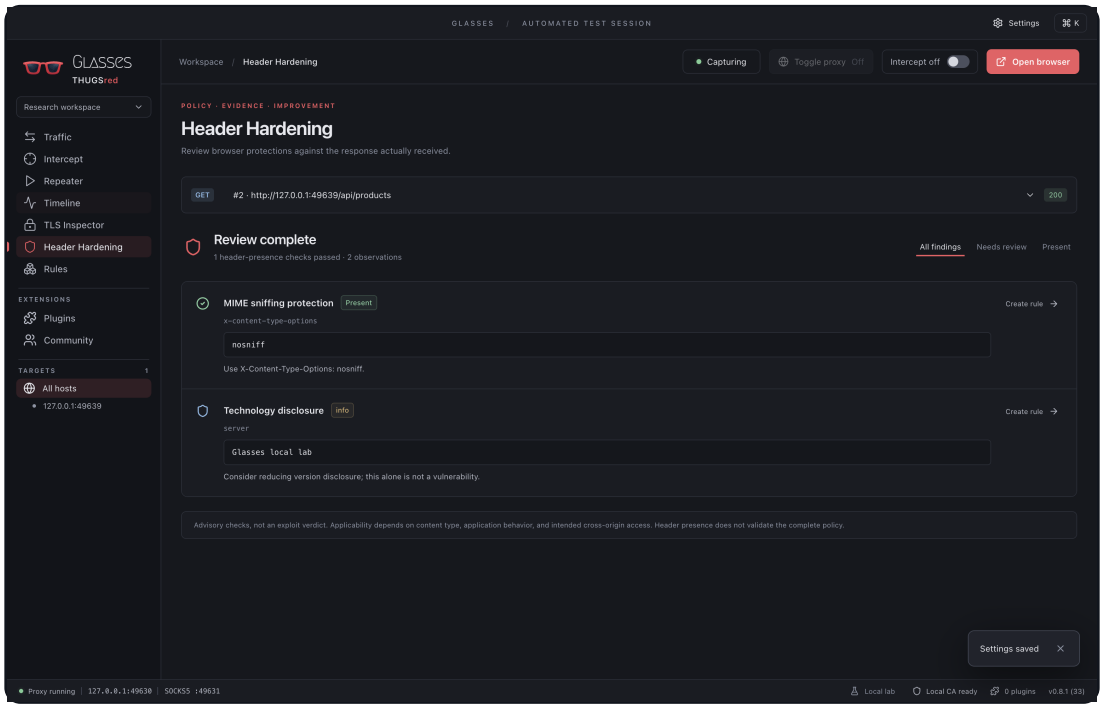
One negotiated connection

Protocol, cipher, ALPN and handshake duration describe that connection. This is not an exhaustive cipher scan or vulnerability assessment. The illustrated local TLS lab uses the active test CA and is explicitly trusted for this demonstration.

RESPONSE POLICY

Header Hardening and rules

Use response evidence to review browser protections, then test a specific policy change.



Header findings are advisory and grounded in the selected response, including lab endpoints with absent policies.

Review the findings

The checks cover CSP, CORS, HSTS, nosniff, framing, referrer/permissions/isolation headers, cookie flags and technology disclosure. Select a captured response to inspect the observed values and advice.

Test header changes

Rules can set or remove a request/response header for a literal URL-substring match. Create a rule from a header context menu or in Rules. Enable it, repeat the exchange, and inspect the visible applied-rule provenance.

Treat results as context

A missing policy is a review item, not proof of a vulnerability. Public APIs may intentionally allow broad CORS; cookies differ by purpose. CSP checks are heuristics, not a full policy parser.

Keep the evidence clear

Rules modify actual forwarded traffic, not only the display. Transport-managed headers cannot be rewritten through the rule API. Bypassed destinations skip rules. Plugin-provided checks are attributed to the extension.

NETWORK ROUTING

The system proxy toggle

Switch between Glasses and your saved network configuration without dismantling the research workspace.

Turn it on

On macOS, click **Toggle proxy** next to Start proxy. Glasses starts its listeners if needed and configures HTTP, HTTPS and SOCKS on enabled services in the current network location. macOS may request authorization.

Corporate setup comes first

If an existing manual proxy or PAC configuration is detected, Glasses requires an upstream relay before replacing it. Use **Show current macOS proxies** in Settings as a reference. PAC evaluation and credential import are not automatic.

Turn it off

Click again to restore the saved proxy dictionaries and protocol-enabled state, including prior PAC, bypass and authentication settings. The listeners remain running for clients configured manually. Stop/quit also restores managed system settings.

Research browser behavior

Switching off returns open research sessions to system routing and closes pooled connections. Reload to create a fresh connection. On a fresh launch, Open browser explicitly uses Glasses even when global routing has never been enabled.

Recovery and conflicts

A recovery journal is saved before changing settings. After an unclean exit, restoration is attempted next launch. If another application changed a service, Glasses preserves its changes and releases settings still owned by Glasses. A recovery report is saved. If the OS refuses restoration, Quit anyway closes the app and keeps the journal for next launch.

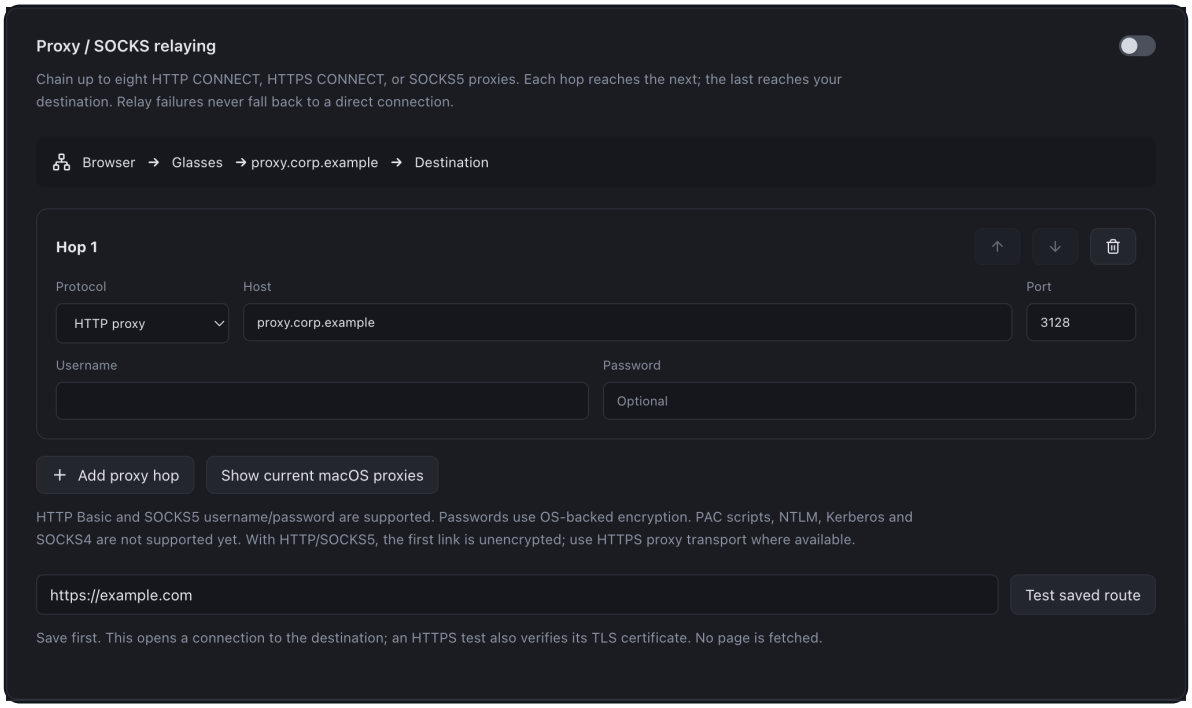
Scope of the switch

This is a system-proxy convenience, not a VPN. Explicitly configured apps can ignore it; new services/locations need another toggle cycle. Windows/Linux use manual proxy setup in this release. Tests cover restore/rollback, conflicts and shutdown failures. A real leftover macOS proxy configuration was restored successfully; fresh takeover still needs interactive acceptance.

CORPORATE NETWORKS

Proxy / SOCKS relay chains

Put Glasses in front of a mandatory corporate proxy, or build a deliberate sequence of upstream hops.



Example configuration, relay disabled until enabled and saved. Use your real corporate endpoints.

Configure in order

In Proxy & interception, add a hop, select HTTP, HTTPS or SOCKS5, and enter host/port. Add credentials if needed. Use the arrows to reorder up to eight hops. Enable relaying and save. The diagram follows connection order.

Test the saved route

Use an HTTPS test destination. The test opens the connection and validates TLS without fetching a page. HTTP destinations use absolute-form forwarding at the last HTTP(S) proxy. Intermediate links and HTTPS targets use CONNECT.

Authentication and privacy

HTTP Basic and SOCKS5 username/password are supported. Relay passwords use OS-backed encryption and are not returned to the renderer. The first HTTP/SOCKS5 link is unencrypted; use HTTPS proxy transport where available.

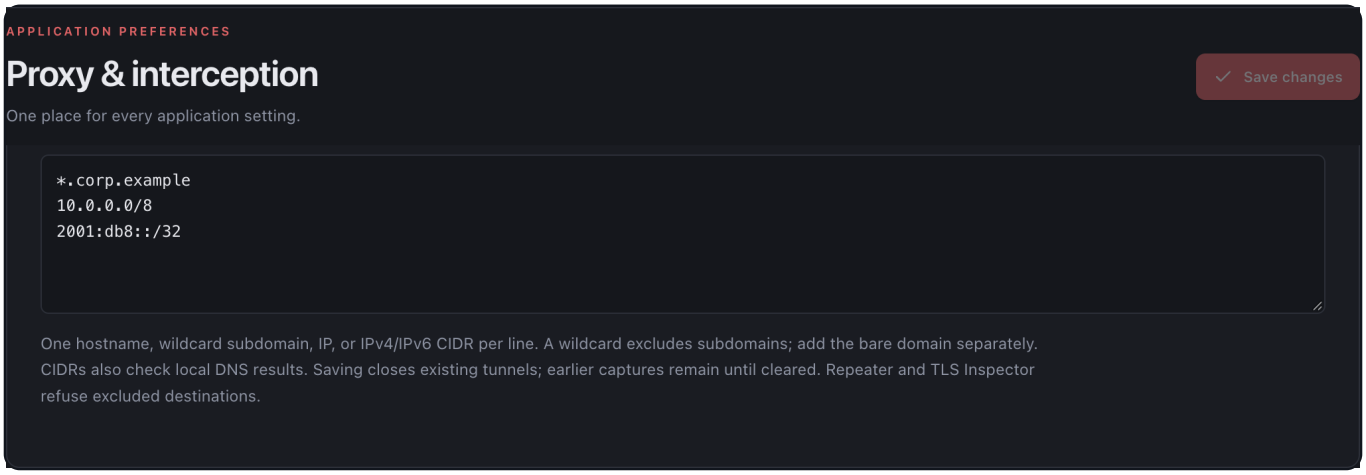
Explicit boundaries

Relay failure never falls back to direct access. PAC scripts, NTLM, Kerberos and SOCKS4 are not yet supported. A relay is also used for excluded traffic, Repeater, TLS inspection and WebSocket upgrades. Remote SOCKS DNS and local CIDR checks can differ.

CONFIDENTIAL DESTINATIONS

Privacy bypass lists

Exclude sensitive hosts from interception and history while retaining the network route they need.



Examples only: *.corp.example, 10.0.0.0/8 and the IPv6 documentation range 2001:db8::/32.

Write precise entries

Use one entry per line: a hostname, `*.corp.example`, an IP, or an IPv4/IPv6 CIDR. Blank lines and lines beginning with # are ignored. Up to 500 entries are accepted.

Understand matching

A literal hostname is exact. A wildcard matches subdomains only; add the bare domain separately. IP ranges also check locally resolved hostname addresses. If a CIDR check cannot resolve, the connection is withheld.

What bypass changes

Excluded HTTPS CONNECT stays an opaque tunnel, preserving the origin certificate. Excluded HTTP/SOCKS traffic is forwarded without capture, interception, rules or plugin visibility. The saved corporate relay still applies.

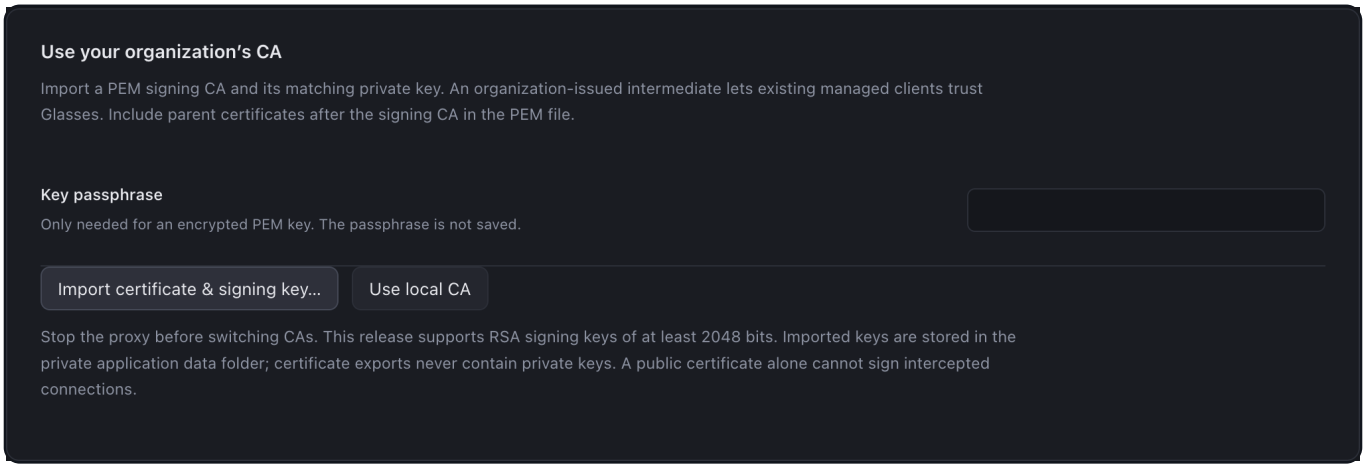
Apply before browsing

Saving closes existing tunnels and cancels pending work. Repeater and TLS Inspector refuse excluded targets. Older captures and exports remain until you clear/delete them. For remote split DNS, list sensitive hostnames explicitly; local CIDR checks cannot know a relay's different DNS answer.

IDENTITY AND TRUST

Organization CA and trust

Use a dedicated local CA or an organization-issued signing CA already trusted by your managed clients.



Certificate import is an explicit operation while the proxy is stopped. No private key is shown in the UI.

Import the signing material

Stop the proxy. In Certificates, enter a key passphrase if needed, then choose **Import certificate & signing key…**. Select the PEM CA file first and its matching PEM private key second. Place parent certificates after the signing CA in issuer order.

Validation and support

Glasses checks CA/signing usage, validity, key match and supplied issuer signatures. The first release accepts RSA keys of at least 2048 bits. Leaf validity never extends beyond the signing CA. ECDSA and hardware-backed signing are deferred.

Trust is a separate action

A public CA alone cannot sign intercepted connections. Clients need to trust the signing chain. Glasses never installs OS trust automatically. The research browser narrowly accepts active-CA signatures with valid names and dates.

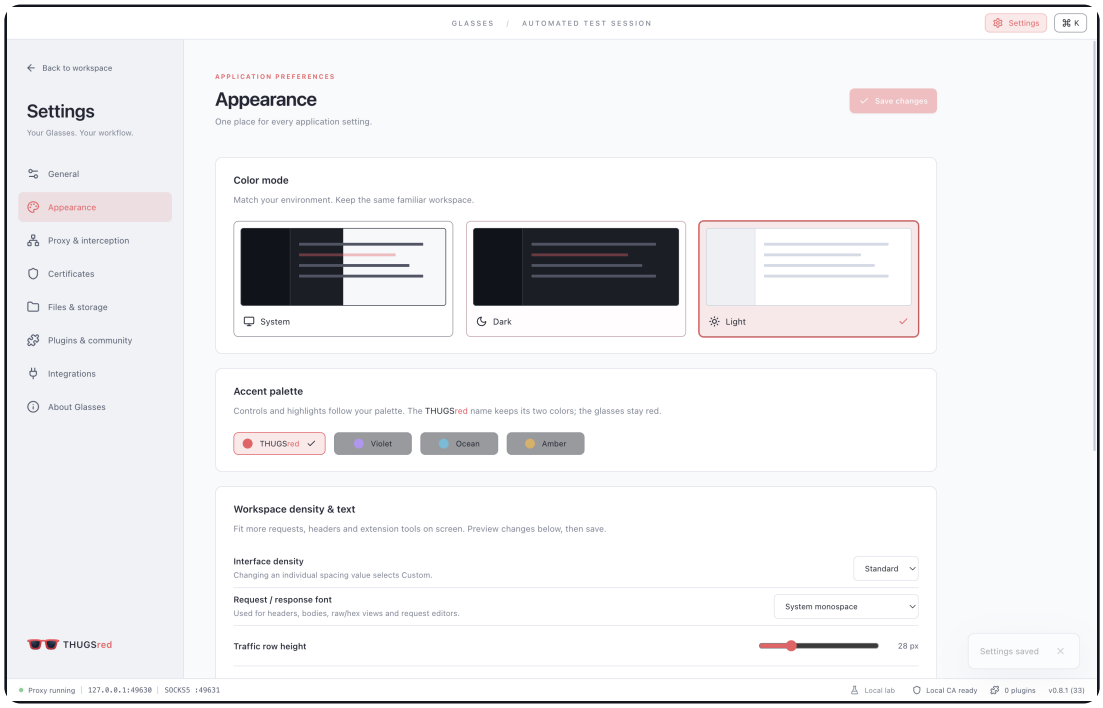
Protect and recover

Imported keys are stored with private file permissions, unencrypted at rest; the unlock passphrase is not saved. Export CA certificate returns public material only. **Use local CA** removes the imported copy and returns to the original installation CA.

PREFERENCES AND FILES

Themes, storage and exchange

Keep the main workspace focused while tailoring presentation, limits and evidence handling in Settings.



Light mode uses the same dedicated Settings layout and keeps the product's red branding.

Appearance

Choose System, Dark or Light and an accent palette. The glasses and THUGSred suffix stay red. Workspace density & text offers Standard, Compact or Custom row heights, header padding, text size and line height. Choose system mono, bundled JetBrains Mono or an installed Nerd Font. Preview, then Save changes.

Session exchange

Save a native Glasses session to reopen captured work. Export HAR for exchange with other tools. Import accepts native sessions and HAR, normalizing some metadata; imported HAR does not restore original per-phase timing.

Storage controls

Files & storage selects the default export folder, reveals application data, sets the entry limit and capture budget. Normal captures remain in memory. Exports can contain cookies, tokens and bodies, so choose their destination deliberately.

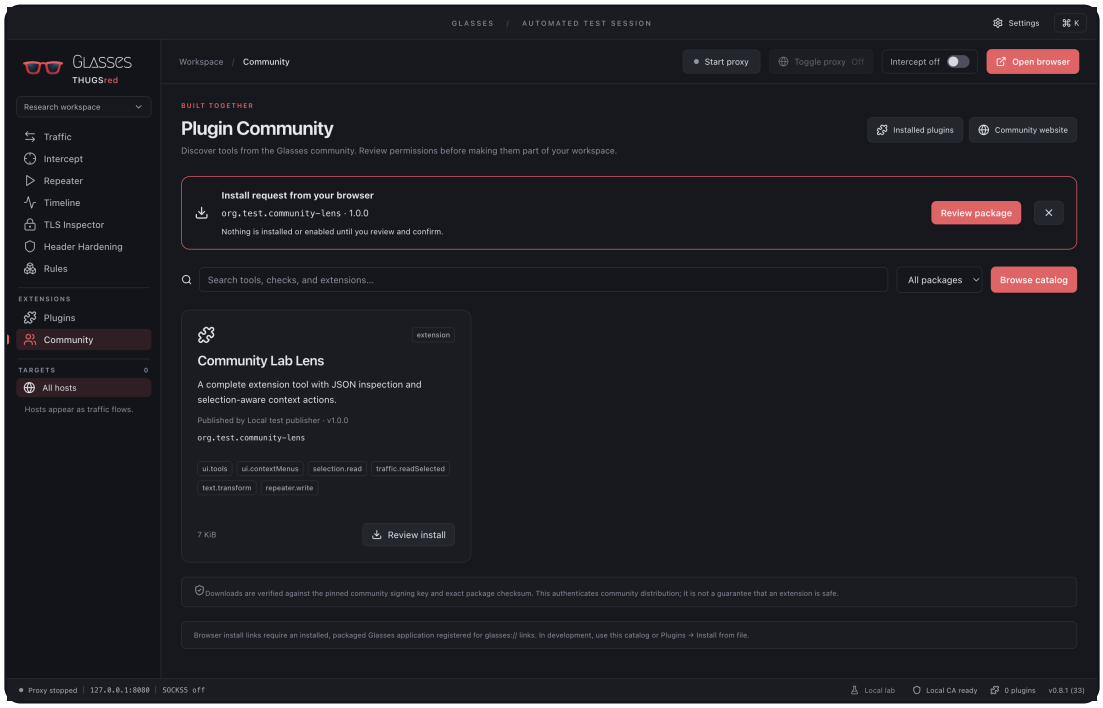
Important defaults

1,000 entries; 2 MiB per captured body; 30-second network deadline; 120-second interception deadline. Port changes need a listener restart. Toggle managed system routing off before changing ports. Unsaved edits can be saved or discarded when leaving Settings.

EXTENSIONS

Install a community tool

Review the package and its capabilities before installation. Community connections are off until you opt in.



Community browsing with a local signed test package. This is a test catalog, not a live-service claim.

Browse and review

Open Community, choose **Connect and browse**, then search or filter. **Review install** verifies the signed descriptor, checksum and manifest. Read the capabilities, then choose Install disabled or Install & enable.

Manage and update

Enable, disable or remove contributions in Plugins. A newer community version requires another review; new capabilities are marked. Local packages cannot be replaced silently. Offline: use Plugins > Install from file with a JSON manifest.

Browser install links

A glasses://install link opens an install request. It never downloads or activates code by itself. Choose Review package to proceed. Packaged apps declare the URL scheme; browser/OS association needs platform acceptance.

Understand trust

Signatures authenticate distribution, not code safety. Sandboxing and declared capabilities still apply. Install screenshots use a signed local fixture. All four bundled examples subsequently passed production-signature and installation checks in an isolated packaged macOS profile. OS browser-link handoff still needs manual acceptance. See docs/COMMUNITY.md for trust and network details.

EXTENSION DEVELOPMENT

Build your own extension

API v1 uses a bounded JSON manifest with declarative contributions and isolated HTML/CSS/JavaScript.

Declare the surface

Set apiVersion: 1, an id, name, semver version, author and description. Declare capabilities before adding tools or contextMenus. A manifest is limited to 256 KiB, with up to 10 tools and 30 contextual commands.

Choose capabilities

UI: ui.tools, ui.contextMenus.
Context: selection.read, traffic.readSelected.
Actions: text.transform, repeater.write.
Declarative analysis: request.headers, response.headers, response.analyze.

Receive context

Use `glassesPlugin.onContext(callback)`. The object identifies command, originating view and toolId. selection.read permits selected text; traffic.readSelected permits the related record with base64 captured bodies. The last context is also window.glassesContext.

Treat traffic as hostile

Use textContent to display untrusted strings. Do not turn captured HTML into executable UI. Bundle code/styles in the manifest; remote imports are denied. Host messages are source-checked, rate-limited to 20/second, with text results up to 1 MiB.

Prepare a handoff

With repeater.write, call `glassesPlugin.sendToRepeater(draft)` using url, method, headers, body and bodyEncoding. The host validates the draft and the researcher clicks Send. With text.transform, showText opens a result preview.

Develop from the SDK

The repository's plugin-sdk/README.md contains the complete manifest and bridge contract, surfaces and runnable examples. UI frames are opaque-origin sandboxes. CPU/memory quotas, arbitrary proxy hook workers, packaged assets and persistent plugin storage are not in v1.

SECURITY MODEL

Treat hostile content as data

Glasses intentionally handles untrusted traffic. Its research capabilities are separated from application authority.

Renderer and browser

The application UI is local, sandboxed, context-isolated and has no Node integration. Captured HTML/SVG/JavaScript is rendered as text. A separate research browser can run site JavaScript without receiving the application IPC bridge.

Privileged operations

The main process validates the calling window, top frame and origin as well as action payloads. There is no generic shell or arbitrary file-read bridge. Native file dialogs provide deliberate import/export boundaries. The About action opens one fixed HTTPS URL.

Bounded processing

Header construction rejects injection characters. Capture limits, request/interception deadlines and a bounded decompressor limit resource use. Operational logs contain metadata rather than traffic bodies, URLs, keys or credentials.

Network controls

Upstream TLS validation remains enabled for ordinary traffic. Relay failures do not use direct access. Private destinations are excluded before capture and MITM. Local listeners are loopback-only but are not authenticated against other programs on the same machine.

Extension isolation

An opaque frame, restrictive CSP and capability-checked messages separate extension code from the core. These controls do not prevent every resource-exhaustion or misleading-interface problem. Disable a misbehaving extension.

Release posture

Packaged fuses disable RunAsNode, Node options and Node inspection, require ASAR and enable supported integrity checks. Independent security review, signed/notarized distribution, updates, encrypted evidence vaults and enterprise administration remain work. See SECURITY.md for the full threat model and source references.

DEVELOPER REFERENCE

Build and release reference

The repository contains the application, proxy core, extension SDK, tests and native packaging configuration.

Run from source

Use Node.js 24 LTS and npm. Run `npm ci`, then `npm run dev`. On macOS the native helper build needs Xcode command-line tools. For production UI: run native, build, then start.

Validate

`npm run check`: build and core tests.
`npm run test:ui`: Electron workflow and screenshots.
`npm run security:check`: build/tests plus dependency audit.
`npm run test:packaged`: local hardened ARM64 package smoke.

Package

Versions use major.minor.patch (build). About can copy the app identity. PDF footers identify the documentation edition and covered app. Use release:all from the Mac checkout. macOS builds locally; Windows uses a local Docker/Wine worker; Linux uses a disposable Docker/SSH worker. Packages and SHA256SUMS return to release/. See docs/BUILDING.md for worker setup.

Verified versus configured

51 core/security/network/version tests pass. Desktop, community, recovery and hardened ARM64 package smokes pass. The native helper restored real leftover macOS settings. Linux and Windows packages were built for x64 and ARM64. Windows/Linux GUI acceptance is separate.

Protocol boundaries

HTTP/1.1 inspection; HTTPS MITM; SOCKS5 TCP metadata; WebSocket upgrade passthrough. No HTTP/2 or HTTP/3 inspection, gRPC semantics, SOCKS UDP or WebSocket frame editing. The TLS tool observes one negotiated connection.

Shipping work

Packages are development builds without distribution signing or notarization. Intel macOS was not run on Intel hardware. The repository uses CC BY-NC 4.0 with additional workplace/consulting permission; read LICENSE before redistribution.

KEEP MOVING

Troubleshooting and glossary

Start with the route, then trust, then the specific request. Keep the version and a minimal reproduction with any report.

No traffic appears

Check that the listener is running and the client actually uses it. Toggle proxy only affects system-aware clients. Explicit browser proxy settings can override it. Review protocol and scope filters, then the privacy bypass list.

Connection or relay errors

Confirm hop order, hostname, port and credentials. Save before Test saved route. HTTP 407 usually means proxy authentication is needed; this release supports Basic, not NTLM/Kerberos. A relay may disallow CONNECT to a requested port. There is no direct fallback.

HTTPS fails

Check the upstream certificate with TLS Inspector. For intercepted traffic, verify client trust in the active signing chain. For pinned clients, use an appropriate bypass entry. Confirm CA validity and that the import includes a matching RSA signing key.

Requests remain paused

Review Intercept and its URL/phase filter. Forward, drop, or turn Intercept off. Deadline expiry drops the message. Large responses may have already streamed without a whole-body pause.

Restoration conflict

Glasses preserves externally changed endpoints and releases remaining settings it owns. A conflict does not block closing. If macOS rejects the write, use Quit anyway or keep working; the recovery journal remains for the next launch. After an unclean exit, check network settings if connectivity is affected.

Terms

MITM: decrypting and re-signing a client's HTTPS connection for inspection.
CA: certificate authority used for signing.
Relay/hop: a proxy in the upstream route.
CIDR: an IP network/prefix such as 10.0.0.0/8.
HAR: HTTP Archive exchange format.

Further reference

Project: README.md, SECURITY.md, docs/RELEASE_NOTES.md and plugin-sdk/README.md.
Electron security guidance
Apple network preferences API
SOCKS5 protocol specification
Official Glasses home